College of Engineering and Computer Science, VinUniversity
COMP2040 Operating Systems – Fall 2026

Group Project and Suggested Themes

In the labs we write small, separate simulators: a CPU scheduler in one week, a page-replacement algorithm in another, a disk scheduler later on. In a real operating system these mechanisms run at the same time and depend on each other. A process that page-faults stops using the CPU and joins a disk queue; the disk completion is an interrupt that puts the process back on the ready queue; a process blocked on a semaphore is invisible to the scheduler until another process signals. The project this semester is to build **one integrated kernel simulator** in which all of this happens together, and then to use it as an instrument for experiments. The simulator, called `ossim`, is tick-driven and written in C. A workload file describes processes as sequences of CPU bursts, I/O, disk requests and semaphore and resource operations, and the command line selects the scheduling, page-replacement and disk-scheduling policies, the memory size and the disk parameters.

The project has two parts. The **core**, worth 90% of the project mark, is the same for every group: starting from the scaffold on Canvas, you implement the remaining CPU schedulers, deadlock detection, page replacement, disk scheduling and a workload generator, verify them against hand-computed cases, and report experiments run on the simulator. The details are in the Project Guideline at the end of this document. The **theme**, worth the remaining 10%, is where groups go in different directions. Once the core works, each group extends the simulator with one additional mechanism and uses one experiment to study what that mechanism actually changes.

A few suggested themes are listed below. Any group may choose any theme, and more than one group may choose the same theme. You are encouraged to discuss possible directions with the instructor and the TAs, and to choose something you find interesting. A theme of your own is also welcome if it is of similar size; please discuss it with us before Week 12.

1. **Working Set and Load Control.** When the total demand for frames exceeds physical memory, a system with global replacement can spend most of its time servicing page faults. The working-set model gives the kernel a way to see this coming. This project tracks the working set of every process over a window Δ and suspends a process when the working sets no longer fit in memory, resuming it when there is room again. Can working-set-based load control prevent thrashing in the simulator, and how sensitive is the result to the choice of Δ and of the process to suspend? The working-set example in the textbook ($\Delta = 10$) is a natural first test.

2. **Local versus Global Frame Allocation.** Under global replacement a process with poor locality can take frames away from every other process. Local replacement, with equal or proportional allocation, keeps the damage inside the process that causes it. When one badly behaved process runs next to well-behaved ones, which allocation scheme gives the best CPU utilization, and which one is fairest to the other processes? The proportional-allocation example in the textbook (62

frames, processes of 10 and 127 pages) is a convenient test. Groups that want to go further may add a TLB and report its hit ratio and the effective access time.

3. **Priority Inversion and Priority Inheritance.** A high-priority process waiting for a lock held by a low-priority process can be delayed for a long time by medium-priority work, as happened on the Mars Pathfinder in 1997. This project makes a semaphore used as a lock remember its holder and implements priority inheritance on top of the priority scheduler. How does the waiting time of the high-priority process grow with the CPU demand of the medium-priority one, with and without inheritance? The classic three-process example, worked out by hand as a Gantt chart, is the expected test. Aging against starvation is a natural addition.

4. **Proportional-Share Scheduling.** Lottery and stride scheduling give each process a share of the CPU in proportion to its tickets, instead of deciding by priority or burst length. This project adds one or both of them to the simulator and compares them with MLFQ. How quickly does each process's share of the CPU approach its share of the tickets, and how does the randomness of lottery scheduling compare with the deterministic behavior of stride scheduling over short and long runs? A stride schedule for tickets $3 : 2 : 1$ can be traced by hand for the first quanta; lottery runs should be repeatable with a fixed seed.

5. **Dirty Pages and Write-Back.** Evicting a modified page costs a disk write before the new page can be read in, so the real cost of a page fault depends on how much of memory is dirty. This project extends the reference streams with writes, adds a dirty bit, and makes the eviction of a dirty page issue a write-back request first. How do page-fault service time and CPU utilization change as the fraction of writes grows, and does enhanced second chance reduce the number of write-backs compared with Clock? A short reference string with a few writes, counted by hand under FIFO, is a good first test.

6. **Striping across Two Disks.** When memory is scarce, the disk becomes the bottleneck of the whole system. RAID-0 striping spreads requests over several disks so that they can be served in parallel. This project generalizes the disk module to two disks, each with its own head, queue and scheduler, under a stripe mapping of your choice. How much does a second disk shorten page-fault service and turnaround under memory pressure, and does the choice of disk scheduler still matter once the load is split? The textbook request queue, split according to your mapping, gives a test that can be checked by hand.

7. **Deadlock Avoidance with the Banker's Algorithm.** Detection lets deadlocks happen and deals with them afterwards; avoidance refuses any request that could lead to one, even when the resources are free. This project adds maximum claims to the workload format and runs the safety algorithm whenever a resource is requested. How much throughput and waiting time does avoidance cost compared with detection, and how much worse does it become when processes overstate their maximum claims? The textbook example with five processes and resources (10, 5, 7) is the expected test.

8. **Deadlock Recovery.** Detecting a deadlock is only half of the job; the system then has to break it. This project terminates or rolls back a victim whenever `deadlock_check()` finds a deadlock, with at least two victim-selection policies and a guard against choosing the same process every time. How does the detection interval trade the time a deadlock goes unnoticed against the work lost to rollback, and which victim policy loses the least work? The detection example of Section 8.6.2 in the textbook is the expected test.

Important Dates:

- Checkpoint 1 (processes, scheduling, synchronization): Week 7
- Checkpoint 2 (memory and storage) and theme choice: Week 12
- Last change of theme: Week 13
- Final submission and demo: Week 15

Project Guideline

1. **Core Simulator**
   - Groups of 3–4 students. Start from `ossim_scaffold_v2.zip` on Canvas; `README.md` explains the code and `CHANGES.md` lists what changed since the first release. Groups that started from the first version should merge these changes.
   - Checkpoint 1: SJF (non-preemptive), SRTF, preemptive priority with round-robin among equal priorities, and a three-level MLFQ (quanta q, $2q$, $4q$, demotion on quantum expiry, periodic boost). Deadlock detection using either the wait-for graph or the multi-instance detection algorithm. A test for each algorithm with expected values computed by hand, and a two-page design note.
   - Checkpoint 2: LRU and Clock replacement, and SSTF, SCAN and C-LOOK disk scheduling, verified against the textbook cases (FIFO 15 and LRU 12 faults with three frames; head movement 640, 236, 236 and 322 for FCFS, SSTF, SCAN and C-LOOK). A workload generator in any language. An updated design note that also states your theme and a short plan for it.
   - The textbook checks are already in `tests/run.sh` and show SKIP until the corresponding policy is implemented. By each checkpoint, the checks for that checkpoint should pass.

2. **Project Theme**
   - Choose one theme from the list above, or a theme of your own agreed with us. The choice can be changed until Week 13.
   - The extension is switched on by a command-line option, so that a run without the option behaves exactly as before.
   - Include at least one test with a result computed by hand, one experiment on the theme in the report (it counts as one of the three experiments below), and a short part of the demo showing it.
   - A small extension that works and is well explained is worth more than a large one that only partly works.

3. **Final Submission and Demo**
   - Due Week 15: the complete simulator, the workload generator, the test suite, and the report, submitted via Canvas together with the link to your git repository.
   - Report length: 8 to 10 pages. Materials expected: design of the simulator, validation against hand-computed cases, and at least three experiments, each with a question, a method, a plot or table, and a discussion of the results.
   - Possible experiments for the core: waiting and response time of the six schedulers under varying I/O intensity; time quantum versus context switches and response time; page-fault rate and CPU utilization as the number of frames shrinks, and the onset of thrashing; Belady's anomaly for FIFO but not LRU;

head movement and disk waiting time of the four disk schedulers under random and localized requests.

- Each group gives a ten-minute demo followed by questions to each member. Any member may be asked to explain any part of the simulator.
- Each member submits a confidential peer assessment together with the final report.

4. **Rules**
   - The simulator must build with `make` using `gcc` or `clang` with the flags `-std=c99 -Wall -Wextra -Werror` on Linux, macOS or WSL, without external libraries.
   - Work in a shared git repository with regular commits from every member. The commit history is part of the evidence of individual contribution.
   - AI assistants and online material may be used to understand concepts and to debug, but every line submitted must be understood by the person who committed it and by the group. Code taken from elsewhere must be cited in the report.
   - Groups may choose the same theme, but code is not shared between groups.
5. **Project Grading (20 marks)**
   - Checkpoint 1: 3 marks
   - Checkpoint 2: 2 marks
   - Final code: 7 marks (completeness, correctness, tests, code quality)
   - Report: 3 marks (validation, experimental design, plots and tables, discussion)
   - Demo and Q&A: 3 marks
   - Theme: 2 marks (1 for the extension working and tested, 1 for the experiment and discussion)
   - The group mark is adjusted per member by up to $\pm 20\%$ based on the commit history and the peer assessment.